Inspectra AI

Integrated image restoration and industrial defect inspection



Final technical project report

October 2026
Windows desktop application for SEM and industrial inspection images

Project scope

Image restoration, 2x super-resolution, pixel-level defect detection, operator decisions, and collective batch reporting form one local workflow. This report documents the implemented application and the earlier research evidence that informed it.

Report basis

The supplied Drift-Sense presentation, AI Restoration Studio presentation, and restoration project report are the historical sources. The current application source, installer script, dependency inventory, and functional test receipt establish the implemented release. Earlier benchmark results retain their original scope and limitations.

Executive summary

Inspectra AI brings enhancement and defect inspection into a single operator-facing application.

Implemented outcome

The release integrates three restoration checkpoints, four super-resolution checkpoints, and fifteen defect checkpoints. Operators select defect types and either one model family or automatic model selection. The application displays four image views, records post-inspection decisions, and exports a concise two-page batch summary. A Windows installer includes the model weights and required runtime libraries. [R4-R6]

Research foundation

The restoration study reports 4,158 grayscale SEM sources and separate denoising and super-resolution benchmarks. The Drift-Sense study contributes COCO segmentation, multi-architecture experimentation, and interpretable defect localization. These streams provide complementary foundations, but their historical performance figures do not establish current end-to-end assembly-line accuracy. [R1-R3]

Evidence and conclusion

The installed build passed a smoke test covering 22 model forward passes, a synthetic inspection, PDF generation, batch rollover, and PDF-only report output. This establishes functional integration. Production defect accuracy, sustained throughput, and closed-loop navigation recovery remain separate validation or integration tasks. [R6]

Report navigation

Pages 3-5 explain the problem, scope, and datasets. Pages 6-11 describe the architecture, models, selection rules, and metrics. Pages 12-15 cover operator interaction, batch handling, storage, and reporting. Pages 16-19 present historical findings, verification, deployment, and limitations.

Problem definition and motivation

Inspection quality depends on both the image presented to the model and the way its results enter the production record.

Image degradation

SEM and industrial acquisition can produce noisy, blurred, unevenly illuminated, or low-resolution images. Weak defect boundaries make visual assessment difficult, while aggressive enhancement may invent plausible detail. The restoration study therefore separates acquisition-noise removal from spatial-resolution recovery and retains the original image for comparison. [R2-R3]

Defect localization

An image-level label alone does not explain where a defect occurs. Pixel segmentation supplies spatial evidence for Blowhole, Crack, Fray, and Break predictions. Connected regions support counts and confidence summaries, although the resulting region count can differ from the number of physical defects. Inspectra AI combines class-specific overlays so operators can compare predictions with the source image. [R1, R4]

Operational record

Assembly-line inspection requires more than an inference button. Each sample needs an accountable worker, batch identity, product identity, and a defined batch capacity. Captured images must remain distinct from completed inspections. A manual quality decision must follow the result, and the exported report must summarize the batch without overwhelming the reader with image appendices. [R4]

Relationship to Drift-Sense

The supplied Drift-Sense material describes segmentation as a foundation for visual navigation-error recovery. Feature extraction, reference matching, position estimation, and closed-loop correction are subsequent integration steps. The current Inspectra AI release performs visual inspection and reporting; it does not command a wafer stage or correct navigation drift. [R1, R4]

Objectives and implemented scope

The project objective is a reproducible local workflow from an inspection image to an accountable batch summary.

Functional objectives

The application accepts uploaded or camera-captured images, restores or upscales them when selected, and detects one or more chosen defect types. Automatic enhancement selection compares available candidates. The detector consumes the latest successful stage. Operators can filter overlays, inspect quality values, and record Accepted, Rejected, or Needs Review after processing. [R4]

Interface and reporting objectives

Four image panels retain original, restored, super-resolution, and combined detection views. Model controls and sample details occupy the left side, while image zoom and fit controls support closer review. A progress bar replaces tile counters. Reports contain batch averages and coverage in exactly two pages, with no image gallery or individual sample appendix. [R4]

Scope boundaries

This release uses local checkpoints and interactive capture. It includes no production PLC trigger, camera calibration procedure, automated reject actuator, or measured line-rate guarantee. It also does not train new models inside the desktop interface. Model-training evidence belongs to the earlier research materials and requires separate datasets and notebooks. [R1-R4]

Requirement	Implemented behavior
Traceability	Worker ID, batch number, and product ID required
Finite batch	Positive sample capacity with automatic rollover
Model choice	Automatic selection or one available model/family
Human review	Decision recorded only after inspection
Portable deployment	Bundled app.exe and Inno Setup installer

Dataset foundation and provenance

The historical restoration and segmentation datasets support different tasks and must remain distinguishable.

SEM restoration dataset

The restoration report records 4,158 readable grayscale JPEG SEM images from the NFFA-Europe MEMS_devices_and_electrodes category. Images were standardized to 1024 x 768. A seed of 2026 created a source-level split of 2,910 training, 831 validation, and 417 test images. Both enhancement tasks reuse those source identities. [R3, p.7]

Paired task construction

The super-resolution benchmark uses bicubic 2x geometry with a 512 x 384 input and a 1024 x 768 target. The denoising benchmark uses same-resolution clean/noisy pairs. Its 4,158 samples contain 2,080 single-operator and 2,078 combined-operator examples. Across both tasks, the study reports 8,316 paired samples. [R2-R3]

Defect segmentation dataset

The Drift-Sense presentation records an audited development subset of 209 images and 299 COCO segmentation annotations. It describes polygon-to-mask conversion, checks for duplicates and annotation inconsistencies, and five experimental segmentation architectures. Those counts describe that earlier audited subset rather than the size of a validated production set. [R1, slides 4 and 6]

Data integrity limits

The restoration report identifies duplicate-group leakage risk and stale per-image parameters after five noise groups were regenerated at stronger settings. It also states that authoritative source licensing details were not supplied. These issues limit reproducibility and redistribution claims even when file-integrity checks pass. [R3, pp.9-11 and 28]

Split	Source images	Share
Training	2,910	69.99%
Validation	831	19.99%
Test	417	10.03%
Total	4,158	100%

System architecture

A staged pipeline separates enhancement, detection, operator review, and persistence.

Input and processing layers

The input layer loads an image or captures a frame. Metadata validation associates it with a sample in the current batch. Restoration produces a same-size candidate output. Super-resolution uses the restored image when that stage succeeds, otherwise the original. Each enhancement stage may run one model, all candidates, or be skipped. [R4]

Detection and display layers

Detection receives the most recent usable image: super-resolution first, restoration second, and original as fallback. The detector produces class-specific regions and confidence values, which feed a combined overlay. A background worker performs inference while the main window displays progress and subsequently updates the panels. Zoom and overlay filters change the view without changing saved detections. [R4]

Persistence and reporting layers

A session-specific SQLite database tracks captured samples and their current report paths. Per-inspection records preserve measurements and supporting images outside the PDF-only export folder. Batch reporting reads the latest saved record for each sample and produces an aggregate PDF. Manual decisions update the sample record and the corresponding completed-batch summary. [R4]

Stage	Primary input	Primary output
Acquisition	Image file or camera frame	Registered sample
Restoration	Original image	Restored image and metrics
Super-resolution	Latest successful image	2x image and metrics
Defect inference	Latest checkpoint image	Regions and confidence
Review and reporting	Saved result and decision	Two-page batch PDF

Image restoration models

The current release exposes MIRNet v2, MPRNet, and NAFNet as alternative restoration candidates.

Restoration role

Restoration aims to reduce degradation while preserving the spatial dimensions needed for inspection. MIRNet v2 combines information across resolutions, MPRNet refines images across progressive stages, and NAFNet uses activation-free restoration blocks. Their checkpoint-specific implementations are integrated into the application rather than imported from separate local architecture files. [R3, p.19; R4]

Single model and automatic operation

Selecting one model runs that candidate. Automatic mode evaluates every usable restoration candidate and selects the highest SSIM when a valid clean reference exists, or the lowest BRISQUE otherwise. A failed candidate generates a warning while remaining candidates may still produce a usable winner. The interface displays the selected result rather than every intermediate candidate. [R4]

Structural fidelity

Visible sharpness is insufficient evidence of improved inspection. Noise suppression can also remove a fine crack, and intensity changes may change segmentation confidence. The original panel provides a fixed visual reference. For scientific comparisons, use aligned clean targets and inspect the downstream effect on labeled defect masks. The release does not infer dimensional metrology from enhanced pixels. [R3, p.28; R4]

Historical distinction

The earlier AI Restoration Studio report selected a compact Restormer adaptation for its fixed denoising pipeline. That historical selection and its benchmark values describe a different deployment. The inspected current model inventory contains MIRNet v2, MPRNet, and NAFNet restoration weights, so this report does not describe Restormer as a bundled current candidate. [R2-R4]

Candidate	Main architectural idea	Current role
MIRNet v2	Multi-resolution feature fusion	Restoration candidate
MPRNet	Progressive stages and attention	Restoration candidate
NAFNet	Activation-free residual blocks	Restoration candidate

Super resolution models

The current application compares four alternatives for 2x spatial upscaling.

Resolution recovery

SRCNN, ESRGAN, RCAN, and SwinIR offer different model capacities and reconstruction behavior. A 2x output doubles width and height, giving four times as many pixels and a 300% increase in pixel count. More pixels do not establish four times as much measured physical information. The software records input and output dimensions so the actual upscaling extent is explicit. [R3-R4]

Candidate behavior

SRCNN provides a shallow convolutional baseline. RCAN uses residual channel attention, ESRGAN uses dense residual generator blocks associated with adversarial training, and SwinIR uses window-based transformer processing. ESRGAN-like perceptual objectives may favor convincing texture, so a visually stronger result requires corroboration before being treated as a real defect boundary. [R3, pp.13 and 28]

Sequential processing

Super-resolution follows restoration when both are enabled and restoration succeeds. With restoration skipped or unusable, it processes the original. Detection then uses the super-resolution result if available. This makes enhancement order deterministic and removes a separate detection-input choice from the operator workflow. [R4]

Historical distinction

EDSR achieved the strongest reported PSNR in the earlier restoration study and belonged to that earlier fixed application. EDSR is not among the four super-resolution checkpoint files included in the current build. Historical EDSR performance therefore informs the research discussion without becoming a claimed result for the current automatic selector. [R2-R4]

Candidate	Architecture family
SRCNN	Shallow convolutional reconstruction
ESRGAN	Residual dense generator
RCAN	Residual channel attention
SwinIR	Window-based transformer

Defect segmentation and localization

Class-specific checkpoints detect Blowhole, Crack, Fray, and Break regions.

Training foundation

The supplied defect presentation describes conversion of COCO polygons into binary masks, grayscale preprocessing, resizing, RGB conversion where pretrained encoders need three channels, normalization, augmentation, and train/validation/test evaluation. Its experiments include Attention U-Net, DeepLabV3+, HRNet, SegFormer, and U-Net++. [R1]

Current architecture coverage

Inspectra AI packages four segmentation families rather than every earlier experimental family. SegFormer uses MiT-B0 transformer features, DeepLabV3+ uses convolutional encoders and multiscale context, U-Net++ uses nested skip connections with a ResNet-34 encoder, and the custom HRNet retains multi-resolution fusion. Current checkpoints cover all four classes for SegFormer, DeepLabV3+, and HRNet, with U-Net++ available for three classes. [R4-R6]

Region interpretation

The software converts predicted masks into connected regions and records confidence and area. A long defect may fragment into multiple regions, while touching defects may merge. Counts are therefore model-region counts, not independently verified counts of physical faults. The combined visualization supports class filtering without discarding predictions from the stored result. [R4]

Operator selection

The interface presents four defect choices instead of a long model-by-class list. Choose All and Clear All simplify selection. A model-family dropdown applies one family to chosen classes where available, or automatic mode compares compatible checkpoints for each chosen class. [R4]

Family	Blowhole	Crack	Fray	Break	Weights
SegFormer	Yes	Yes	Yes	Yes	4
DeepLabV3+	Yes	Yes	Yes	Yes	4
HRNet	Yes	Yes	Yes	Yes	4
U-Net++	No	Yes	Yes	Yes	3
Total					15

Automatic model selection

Automatic selection uses a stated scoring rule and runs every available candidate for the chosen task.

Enhancement ranking

With an aligned clean reference, restoration and super-resolution select the candidate with maximum SSIM. Without a reference, the minimum BRISQUE score wins. References must match output dimensions. BRISQUE uses embedded model data and OpenCV SVM inference, so the app does not depend on the optional cv2.quality module or external BRISQUE files. [R4]

Defect ranking

For a selected defect type, the selector calculates an area-weighted mean confidence over detected regions. If region i has confidence c_i and area a_i, the score equals sum(c_i x a_i) / sum(a_i). A candidate with no detected region receives zero. The winning result is retained independently for each selected defect type. [R4]

Meaning of best

The selected model is best under the implemented scoring rule for that image. SSIM reflects target agreement; BRISQUE estimates perceptual image quality; detected-region confidence reflects a model output. None substitutes for a ground-truth defect benchmark. A high-confidence false positive can win the detection ranking, and confidence scales can differ between architectures. [R3-R4]

Operational tradeoff

Automatic mode prioritizes comparison over minimum latency because all available candidates must run. Single-model mode avoids that comparison cost. If a reference changes after processing, the application can refresh displayed metrics, but automatic model selection must be rerun to choose a winner under the new reference. Candidate failures remain visible in warnings. [R4]

Task	Clean reference	Selection score
Restoration / SR	Aligned and same output size	Highest SSIM
Restoration / SR	No reference	Lowest BRISQUE
Defect class	No labeled runtime target	Highest area-weighted confidence

Image quality and confidence metrics

Metrics describe agreement, appearance, or processing cost, and each requires the correct interpretation.

Reference-based measurements

For normalized grayscale images x and target y with N pixels, MSE = sum((x-y)^2)/N and MAE = sum(abs(x-y))/N. PSNR = 10 log10(1/MSE) when the intensity range is one. A perfect match produces infinite PSNR. SSIM compares local luminance, contrast, and structure with an aligned reference. The app excludes unavailable values from aggregate means and records finite valid counts. [R4]

Reference selection

The original degraded input is not automatically clean ground truth. A same-scene, aligned reference should represent the intended clean image and match the stage output dimensions. A low-resolution original cannot directly serve as a same-size target for a 2x output. Without a usable target, PSNR, SSIM, MSE, and MAE appropriately show N/A rather than a fabricated quality score. [R4]

No-reference and confidence values

BRISQUE estimates image quality without a paired target. Laplacian variance, entropy, and contrast describe image properties but do not alone establish improvement. Mean detection confidence summarizes predicted regions and is not a statistical confidence interval. Historical LPIPS values belong to the earlier restoration study; the current interface does not compute LPIPS. [R3-R4]

Metric	Preferred direction	Reference needed
PSNR / SSIM	Higher agreement	Yes
MSE / MAE	Lower error	Yes
BRISQUE	Lower estimate	No
Sharpness / entropy / contrast	Descriptive only	No
Processing time	Lower at equal workload	No
Detection confidence	Interpret by model and class	No paired runtime target

Operator interface and result review

The layout keeps model and sample controls beside four large result panels.

Interaction design

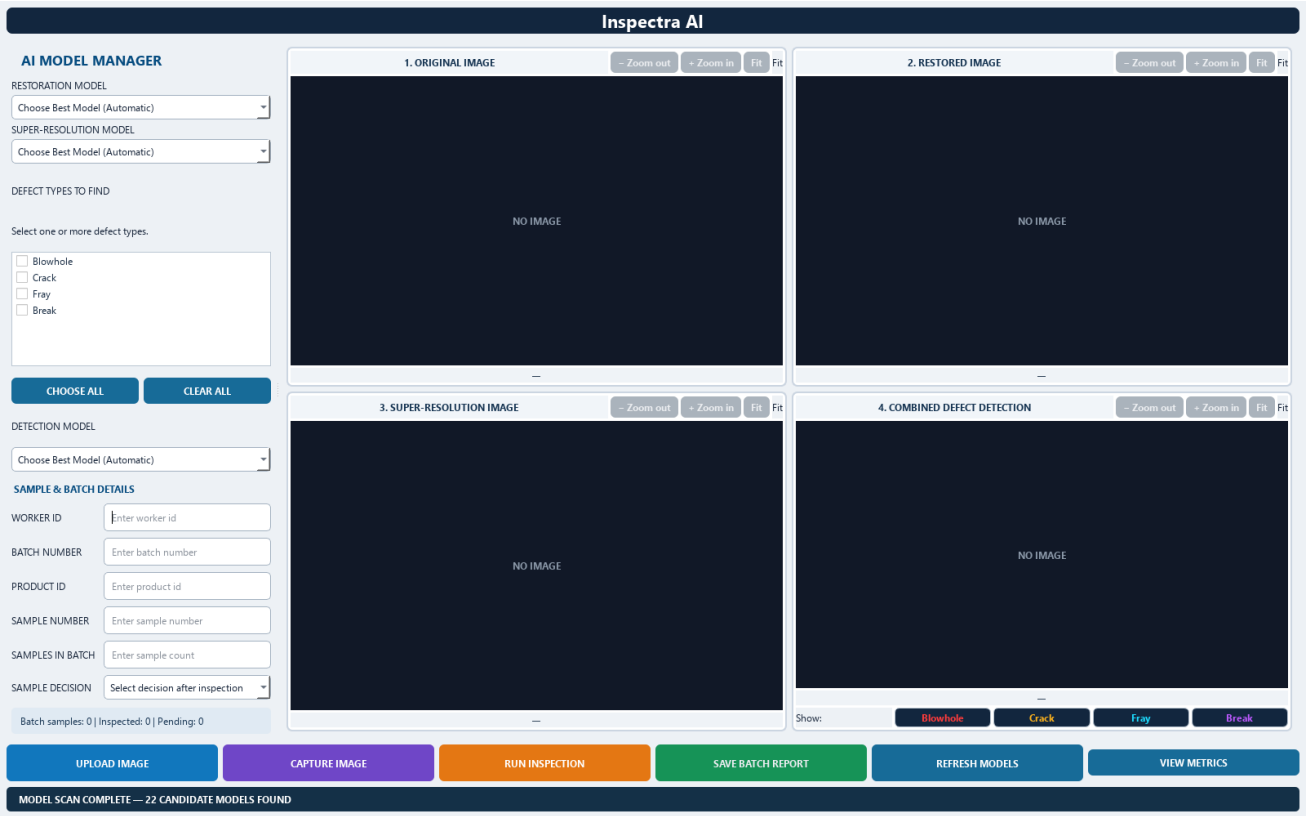
Restoration, super-resolution, defect types, and the detection model occupy the left side. Worker, batch, product, sample, capacity, and decision controls keep the inspection context close to model choices. The four display areas show original, restored, super-resolution, and combined detection images. Separate mask and individual-model panels were removed to preserve review space. [R4]

Review controls

Zoom in, zoom out, Fit, and panning support image examination. Color-labelled defect buttons permit one class or a combination to remain visible. A progress bar communicates processing activity. View Metrics opens quality and confidence details without permanently consuming display space. Saved records retain the underlying detections. [R4]

Decision timing

The decision control remains disabled before and during inspection. After results arrive, the operator can record Accepted, Rejected, or Needs Review. The software does not assign an acceptance outcome from a high confidence score or absence of regions. A completed result can remain awaiting a decision, and the PDF counts that status explicitly. [R4]



Current Inspectra AI source interface with no sample loaded. This view demonstrates layout rather than detection accuracy.

Batch lifecycle and sample accounting

A batch has a configured capacity, and each captured sample contributes at most one latest inspection record.

Registration and capacity

The operator enters required identities and a positive number of samples. Capturing or uploading a sample creates a unique internal sample ID and an image record. The batch receives a fixed capacity once collection begins. Captured samples and inspected samples are counted separately so an unprocessed image is not reported as a completed inspection. [R4]

Reinspection and completion

Running inspection again on the current sample updates its latest saved result and does not create another captured sample. The batch completes when the configured number of samples exist and each has a saved inspection report. A failed sample without a saved result remains incomplete. Capacity prevents additional collection into an already full batch. [R4]

Automatic transition

On completion, the application writes a collective PDF and selects the next available batch identity. A trailing numeric suffix increments while preserving its width, for example LOT001 to LOT002. The next batch retains capacity and restarts sample numbering. A decision made on the just-completed sample after rollover updates its original record and completed PDF. [R4]

Fresh session behavior

Each launch creates a new session and starts counters at zero. Archived data remains on disk but does not resume previous counters. This satisfies fresh-shift counting while retaining internal history. It does not provide a historical batch browser or an automatic cleanup policy. [R4]

Illustrative 100-sample batch	Captured	Inspected	Outcome
First sample collected	1	0	One pending inspection
First sample completed	1	1	Decision may be pending
All results saved	100	100	Completed PDF generated
Next batch selected	0	0	Fresh numbering

Data management and traceability

Report exports and internal inspection records use separate storage locations.

Internal record structure

The application creates a session UUID and a session-specific SQLite database. Each sample stores its batch, worker, product, sample number, capture time, image path, status, and latest report path. Capture folders preserve source images. Inspection folders store outputs, JSON metadata, and supporting text for collective summaries and decision updates. [R4]

PDF-only exports

The runtime outputs directory contains PDFs directly, with no image or database subfolders. Source mode uses outputs beside app.py. Packaged mode uses outputs beside the installed executable, with a LocalAppData fallback if that location is unwritable. Internal data resides under %LOCALAPPDATA%/InspectraAI/data. [R4]

Record consistency

A stable inspection identifier prevents repeated saves from inflating statistics. Reports use the latest report path for each captured sample. Decision changes update the stored record rather than adding a sample. The implementation supports local accountability through metadata; it does not implement authenticated accounts, cryptographic signatures, or a central manufacturing database. [R4]

Retention and recovery

Fresh startup counts do not erase archives. Reports and internal data remain after uninstall. Retention and backups must include source captures, result images, JSON, and SQLite, not just PDFs. Deleting internal records during active work can prevent later collective reports or decision updates. The release has no validated crash-resume workflow for the prior session. [R4-R5]

Artifact	Location or role
PDF reports	Runtime outputs directory
Session database	data/sessions/session ID/assembly_line.sqlite3
Captured images	data/batch_samples/sample ID/captured.png
Result details	data/inspections/inspection folder
Installed program	LocalAppData/Programs/Inspectra AI

Collective reporting design

The operational report is a fixed two-page summary of the current batch.

Page one coverage

The first PDF page presents planned, captured, inspected, pending, and failed counts. It separates Accepted, Rejected, Needs Review, and awaiting-decision totals. Class rows show total regions, samples inspected for that class, mean regions per inspected sample, and mean detected-region confidence. Processing-time totals and means support operational review. [R4]

Page two performance

The second page shows average restoration and super-resolution metrics, valid finite measurement counts, processing coverage, mean width and height scale, and mean pixel-count increase. It omits image appendices and individual sample tables. Historical study results are not inserted into live batch averages. [R4]

Aggregation denominators

Class-region means use only samples inspected for that class. Confidence means use detected regions, so a sample with no region contributes no confidence observation. Metric means exclude missing values; infinite PSNR values are counted separately from finite means. Reinspection replaces the latest sample result. These rules prevent missing references or duplicate saves from distorting averages. [R4]

Interpretation and timing

Save Report generates a snapshot of the current batch, including pending work when present. Completion also triggers a PDF before rollover. A batch with all results may still await decisions, which appear explicitly rather than being silently counted as accepted. The two-page production summary differs from this technical report: one is an operational aggregate, the other documents design and evidence. [R4]

Measure	Denominator or interpretation
Mean regions per class	Samples inspected for that class
Mean confidence	Detected regions of that class
Mean quality metric	Available finite values only
Sample count	Distinct captured identities
Acceptance	Explicit manual decision only

Historical experimental findings

Earlier benchmark results provide research context and retain the qualifications in the supplied sources.

Restoration findings

The earlier study reports EDSR at 29.355 dB PSNR and 0.9819 SSIM for super-resolution. Its compact Restormer adaptation records 20.814 dB PSNR and 0.4085 LPIPS for denoising. Both belonged to that earlier fixed application, while the current release bundles different enhancement candidates. They are not measured results of the current integrated automatic pipeline. [R2; R3, pp.13 and 19]

Comparability limits

The source marks RCAN, ESRGAN, and SwinIR SSIM as originating from a different notebook implementation family. Restormer and MPRNet SSIM also carry an evaluation-code consistency warning. A complete cross-model SSIM leaderboard is therefore not justified. Historical timings describe their source conditions and are not current assembly-line throughput estimates. [R3, pp.13, 19 and 22]

Segmentation findings

The Drift-Sense presentation records provisional SegFormer weighted F1 of 99.76% Blowhole, 99.59% Crack, 96.13% Break, and 83.25% Fray. These are source-reported earlier results and require their label definitions, split audit, and evaluation procedure for reproduction. They do not validate per-image confidence ranking or transfer to a new camera and product. [R1, slide 6]

Earlier experiment	Source-reported result	Scope
EDSR super-resolution	29.355 dB PSNR / 0.9819 SSIM	Historical SEM benchmark
Restormer denoising	20.814 dB PSNR / 0.4085 LPIPS	Historical SEM benchmark
SegFormer defect models	Weighted F1 83.25% to 99.76%	Provisional earlier study

Implementation verification

Functional testing confirms the installed workflow while leaving dataset accuracy as a separate question.

Model and metric smoke tests

The installed self-test discovered 22 supported checkpoint files, loaded each architecture, and performed a small forward pass with finite-output checks. It evaluated the embedded BRISQUE scorer and confirmed SSIM equals 1 for an identical image/reference pair. These checks establish that the bundled definitions and dependencies can execute with the stored weights. [R6]

Workflow checks

A synthetic sample passed through the UI workflow with a selected super-resolution model. The test verified a two-page PDF, completed-batch advance, a recorded inspector decision, and only PDFs in the runtime output directory. The installed EXE matched the build artifact hash. Installation and uninstallation returned success. [R6]

Limits of verification

Synthetic smoke tests do not estimate false positives, defect recall, segmentation overlap, or physical region-count accuracy. They do not establish sustained acquisition, power-loss recovery, or a promised cycle time. The evidence supports software integration and packaging rather than production certification. [R6]

Production validation method

A deployment study should use held-out images from the intended camera and product, source-group splits, and expert-verified masks. Record precision, recall, F1, and IoU with fixed code and thresholds. Measure latency distributions separately for single-model and automatic modes. Compare original-image detection with enhanced-image detection to quantify the downstream effect.

Verified check	Recorded result
Model forward passes	22 checkpoints passed
Identical-image SSIM	1.0
Generated report	Two pages
Completed-batch rollover	Passed
PDF-only runtime output	Passed
Installer and uninstaller	Exit code 0

Packaging and deployment

The release provides both a single Python source and a self-contained Windows installer.

Source distribution

app.py integrates model implementations, reporting, selection, and the icon. Source execution requires its model directory and external packages. Pinned requirements describe the tested Python 3.10 x64 environment: PyQt6, PyTorch and torchvision, NumPy, OpenCV, Pillow, SciPy, scikit-image, timm, transformers, segmentation-models-pytorch, ReportLab, and pypdf. [R4]

Executable packaging

PyInstaller embeds weights, package data, dynamic model imports, and the named Inspectra AI icon. Runtime hooks suppress unused TensorFlow/Flax paths and configure local operation. The one-file EXE extracts resources at launch; reports and internal records use persistent directories outside that extraction location. [R4-R5]

Installer design

Inno Setup installs app.exe and the icon for the current user, creates a Start menu shortcut, offers a desktop shortcut, and registers an uninstaller. It targets Windows 10/11 x64-compatible systems. The tested installer is approximately 1.25 GB because it includes the models and runtime. Separate Python installation is unnecessary for the packaged release. [R5-R6]

Release maintenance

Source changes require rebuilding app.exe before compiling the installer. Keep the .iss, icon, PyInstaller spec, hooks, requirements, and model inventory with the release. Reports and internal data survive uninstall. GPU acceleration requires a compatible source environment; the packaged verification used CPU inference and does not establish CUDA deployment behavior. [R4-R6]

Deliverable	Purpose
app.py and models	Source execution
requirements.txt	Pinned runtime versions
app.exe	Bundled standalone program
InspectraAI.iss	Installer definition
installer.exe	End-user installation

Limitations and development priorities

The current system is a functioning inspection application with explicit scientific and production boundaries.

Image and model limitations

Enhancement can suppress a true feature or introduce an artificial edge. BRISQUE reflects a general perceptual model and may not rank SEM fidelity correctly. Cross-architecture confidence is not calibrated, so automatic selection can favor an overconfident detector. Connected-region counts require comparison with physical annotations before becoming acceptance thresholds. [R3-R4]

Dataset limitations

The historical restoration report identifies duplicates, fixed resizing that may change physical scale, annotation bars that can become shortcuts, unusually severe combined degradation, and stale noise parameters. The defect presentation records annotation/class inconsistencies requiring corrective action before benchmarking. Redistribution terms must come from authoritative source records. [R1; R3, p.28]

Operational limitations

Interactive capture does not provide conveyor synchronization, PLC handshakes, reject control, authenticated operators, or a central audit service. Counts reset on startup, and prior-session recovery is not exposed. Data has no automatic retention cap. Large images and automatic model comparison can increase memory use and cycle time. [R4-R5]

Priorities

First, repair and freeze manifests and evaluation code. Next, benchmark detection on real line images, calibrate outputs, and define class-specific decision criteria with quality engineers. Then measure latency and implement recovery, retention, and integration controls. Drift-Sense reference matching and stage correction require a separate validated extension. [R1, R3-R4]

Priority	Required evidence
Accuracy	Held-out expert masks and fixed metrics
Line rate	Measured latency at target workload
Reliability	Restart, failure, and storage tests
Navigation recovery	Reference matching and stage validation